

MCT-454L: Mobile Robotics

Autonomous Soccer Agent: Motion Modeling and Control in ROS 2



Session: 2022

SUBMITTED TO

Dr. Maria Akram

SUBMITTED BY

Muhammad Haider Ali

2022-MC-45

**Department of Mechatronics & Control Engineering, University of
Engineering & Technology, Lahore, Pakistan**

I. System Architecture

The control system is structured as a modular ROS 2 package composed of four Python modules, each with a single well-defined responsibility. This separation of concerns makes each component independently testable and easy to modify.

Module	Role	Key Exports
control.py	Top-level ROS 2 node; state machine; vision callback; timer loop	BallTrackerNode
apf.py	Artificial Potential Field planner — attractive + repulsive + tangential forces	compute_apf()
utils.py	Math helpers — angle normalization, clamping, frame transform	normalize_angle(), world_to_robot()
packet.py	grSim UDP encoder (protobuf-like, no external dependency)	build_packet()
main.py	Entry point — rclpy init, spin, shutdown	main()

The node subscribes to /ssl_vision_bridge/vision_messages (VisionWrapper) at 50 Hz and sends UDP velocity commands directly to grSim on port 20011. The control loop runs on a 20 ms timer, decoupled from the vision callback, so sensor latency does not stall the controller.

Data Flow

Vision data arrives via the ROS 2 topic and is parsed in _vision_cb(). Ball position, robot pose (x, y, yaw extracted from quaternion), and all opponent and teammate positions are stored as instance state. Obstacle positions are time-stamped and expired after 1 second if not refreshed, preventing ghost obstacles from stale frames. The control loop then reads this shared state every 20 ms, computes velocity commands through the active state handler, and dispatches a UDP packet.

State Machine

The striker operates as a three-state finite state machine:

- **APPROACH_BALL** — navigate to the ball using APF + PD damping; hold for 25 ticks (~0.5 s) before transitioning.
- **ALIGN** — arc behind the ball onto the shoot axis, then rotate to face the goal; declare **READY** after 10 stable ticks within 0.10 rad angular tolerance.
- **READY** — drive into the ball with kicker armed at 6.0 m/s; reset to **APPROACH_BALL** once the ball departs.

Each state includes a ball-moved guard: if the ball relocates beyond the threshold while in **ALIGN** or **READY**, the machine resets immediately to **APPROACH_BALL**, preventing the robot from executing a kick on a ball it has lost.

II. Motion Model

Holonomic Kinematics

The robot is omni-directional (holonomic), meaning it can translate in any direction independently of its heading. Commands are sent in the robot's local frame as three scalars: veltangent (forward), velnormal (lateral left), and velangular (CCW rotation). World-frame velocity vectors from the planner are converted to robot frame using a standard 2D rotation:

$$\mathbf{v_t} = \cos(-\theta) \cdot \mathbf{v_x} - \sin(-\theta) \cdot \mathbf{v_y}$$

$$\mathbf{v_n} = \sin(-\theta) \cdot \mathbf{v_x} + \cos(-\theta) \cdot \mathbf{v_y}$$

where θ is the robot's current yaw from the vision system. This transform is implemented in `world_to_robot()` in `utils.py`.

PD Control — Angular

Heading is controlled by a PD controller. The proportional term drives the robot toward the desired angle; the derivative term suppresses overshoot by opposing rapid angular change:

$$\omega = K_p_{\text{ang}} \cdot e_{\theta} + K_d_{\text{ang}} \cdot (\Delta e_{\theta} / \Delta t)$$

Gains are $K_p = 4.0$, $K_d = 0.05$, with output clamped to ± 6.0 rad/s. Angular error is always normalized to $[-\pi, \pi]$ using `atan2(sin(e), cos(e))` before computing the derivative, preventing derivative spikes at the $\pm\pi$ wrap boundary.

PD Control — Translational

The APF output serves as the proportional term for translational velocity. A derivative term is added using the rate of change of distance error to damp overshoot near the ball:

$$\mathbf{v_pd} = \mathbf{v_apf} + \mathbf{Kd_vel} \cdot (\Delta\text{dist} / \Delta t)$$

When the robot is closing in, $\Delta\text{dist}/\Delta t$ is negative, reducing speed automatically. The PD-adjusted speed is clamped to [0, 2.5 m/s] and re-applied along the original APF direction vector, so only the magnitude is modified, not the obstacle-avoidance heading.

III. Planning Strategy

Artificial Potential Fields (APF)

Navigation uses an Artificial Potential Field approach. The robot is subject to an attractive force toward the target and repulsive forces from nearby obstacles. These are summed and normalized to produce a world-frame velocity command.

Component	Formula	Behaviour
Attractive (near)	$F_{\text{att}} = K_{\text{att}} \cdot (\text{target} - \text{robot})$	Linear pull when $\text{dist} \leq D_{\text{star}}$ (1.0 m)
Attractive (far)	$F_{\text{att}} = K_{\text{att}} \cdot D_{\text{star}} \cdot \text{unit}(\text{target} - \text{robot})$	Capped pull at distance; prevents runaway speed
Repulsive	$F_{\text{rep}} = K_{\text{rep}} \cdot (1/\rho - 1/\rho_0) / \rho^2 \cdot \text{unit}(\text{away})$	Pushes robot away when within $\rho_0 = 0.28$ m
Tangential	$F_{\text{tan}} = K_{\text{tan}} \cdot \text{mag} \cdot \text{dot} \cdot \text{tangent}$	Sidesteps robot around obstacle toward goal

Up to $N=7$ nearest obstacles are considered per tick. Each obstacle's effective radius is reduced by 0.18 m (robot body radius) so the influence field reflects surface-to-surface distance. Repulsive force magnitude is capped at 20.0 to prevent numerical blowup at near-zero distance. The tangential component adds a lateral nudge that steers the robot around obstacles rather than purely away from them, reducing the likelihood of getting trapped in saddle points.

Behind-Ball Arc Maneuver

During ALIGN, the robot must position itself behind the ball on the ball-to-goal axis before facing the goal and kicking. If the robot is not yet behind the ball, a tangential arc waypoint is computed: the robot-from-ball unit vector is rotated 90° in whichever direction reduces the angle to the behind-ball target. The robot is sent to this arc waypoint at half speed (1.25 m/s), skirting around the ball without pushing it. Once the behind-ball check passes, the robot drives to the hold position at `ALIGN_DIST` = 0.22 m behind the ball.

Why Not ORCA?

Optimal Reciprocal Collision Avoidance (ORCA) was evaluated as a candidate for dynamic obstacle handling. ORCA offers provably collision-free paths in multi-agent environments by computing velocity half-planes for each agent pair and finding a velocity inside the intersection. However, several practical barriers made it unsuitable for this project:

Velocity estimation required: ORCA needs reliable velocity estimates for all other agents. In the SSL vision pipeline, robot velocities must be differentiated from noisy position data, introducing lag and noise that destabilize ORCA constraints.

Reciprocity assumption broken: ORCA assumes all agents run the same algorithm and split avoidance responsibility equally. Opponent robots in grSim do not run ORCA, so the robot must take the full avoidance burden — requiring a conservative safety margin that over-constrains the feasible velocity set.

Solver complexity: A proper ORCA implementation requires a linear programming solver (or half-plane intersection) at each control tick per agent pair, adding implementation complexity well beyond the course scope.

Out of course scope: Only potential fields were covered in the curriculum. ORCA is a research-grade algorithm from the UNC GAMMA group, and implementing it correctly under time pressure introduces more risk than benefit.

The APF approach with tangential forces handles the observable obstacle avoidance cases in the SSL environment adequately. The main limitation — local minima when obstacles cluster between robot and ball — is mitigated by the tangential nudge and the arc-around-ball maneuver during ALIGN, which uses a geometrically constructed waypoint rather than relying on the APF to avoid the ball itself.

4. Edge Cases and Robustness

Edge Case	Detection	Response
Ball moves during ALIGN	$\text{dist} > 2 \times \text{STOP_DIST}$	Immediate reset to APPROACH_BALL
Ball moves after READY	$\text{dist} > 2 \times \text{STOP_DIST}$	Abort kick, reset to APPROACH_BALL
No vision data	<code>self._ball</code> or <code>self._robot</code> is <code>None</code>	Control loop skips tick, logs warning
Stale obstacle data	Timestamp expiry > 1 s	Obstacle removed from map
Duplicate vision frames	$\text{dt} < 5$ ms between frames	Ball velocity update skipped
Angle wrap at $\pm\pi$	<code>normalize_angle()</code> on all errors	<code>atan2(sin,cos)</code> prevents derivative spike
Near-zero obstacle distance	$\text{rho} = \max(\text{rr} - 0.18, 0.01)$	Clamp prevents division by zero
APF local minimum	Tangential force + arc waypoint	Lateral push escapes saddle points
Overshoot near ball	Translational D term ($K_d=0.15$)	Speed reduced as distance closes
Robot not behind ball	Dot product with shoot axis	Arc maneuver reroutes around ball

Ball Velocity Estimation

Ball velocity is tracked using an Exponential Moving Average (EMA) with smoothing factor $\alpha = 0.4$. Raw velocity is computed as finite difference of ball position over time. Frames with $\text{dt} < 5$ ms are skipped to avoid division by near-zero time differences producing spurious large velocity

estimates. The smoothed velocity is stored but in the current implementation is available for future use (e.g., intercept prediction) — the state machine uses measured ball position directly.

Obstacle Map Lifecycle

All detected robots (opponents and teammates excluding self) are stored in a dictionary keyed by robot ID with a timestamp. Every vision callback prunes entries older than 1 second. This prevents phantom obstacles from robots that have left the field or stopped being detected, which would otherwise bias the APF repulsive field in incorrect directions.

5. Results and Limitations

Observed Performance

In simulation testing with grSim, the striker successfully:

- Navigated from any starting position to the ball using APF within 3–5 seconds depending on distance.
- Executed the arc-around-ball maneuver to position behind the ball on the goal axis.
- Aligned heading to the goal within 0.10 rad tolerance before kicking.
- Scored goals consistently when no defender was present, and in most trials with a static defender.

Known Limitations

- Dynamic defenders moving directly into the robot's path can temporarily trap the APF in a local minimum before the tangential force resolves it. Increasing RHO_0 (influence radius) mitigates this at the cost of wider detours.
- The ALIGN arc maneuver can be slow when the robot starts directly in front of the ball (opposite side from goal), requiring a near-180° arc. A larger ALIGN_DIST would reduce arc time but risks pushing the ball on approach.
- Ball velocity is estimated but not yet used for predictive interception. Adding lead compensation would improve performance against a moving ball.

Parameter Summary

Parameter	Value	Purpose
MAX_VEL	2.5 m/s	Maximum translational speed
MAX_OMEGA	6.0 rad/s	Maximum angular speed
STOP_DIST	0.15 m	Proximity threshold to ball
ALIGN_DIST	0.22 m	Hold distance behind ball
KP_ANG / KD_ANG	4.0 / 0.05	Angular PD gains
K_ATT / K_REP	4.0 / 6.0	APF attractive/repulsive gains
RHO_0	0.28 m	Obstacle influence radius
K_TAN	3.0	Tangential sidestep gain
KICK_SPEED	6.0 m/s	grSim kick speed
HOLD_TICKS / ALIGN_TICKS	25 / 10	Stability dwell counters

References

grSim Simulator: <https://github.com/RoboCup-SSL/grSim>

ssl_ros_bridge: https://github.com/SSL-A-Team/ssl_ros_bridge

Khatib, O. (1986). Real-time obstacle avoidance for manipulators and mobile robots. International Journal of Robotics Research, 5(1), 90–98.

van den Berg, J. et al. (2008). Reciprocal velocity obstacles for real-time multi-agent navigation. ICRA 2008.